0＃新規フォーマットファイル作成モーダル UI仕様書

1. 概要

1.1 コンポーネントの目的

新規フォーマットファイル作成モーダルは、フォーマットフォルダまたはサブフォルダ配下に新しいファイル（XMLまたはMAIML）を作成するためのモーダルダイアログです。ファイル名の入力、バリデーション、テンプレートタイプの自動判定などを行います。

1.2 使用場面

- **表示場所**: フォーマット管理画面のモーダルオーバーレイ
- **トリガー**:「新規ファイル作成」ボタンのクリック
- **主な用途**:
  - XMLテンプレートファイルの新規作成
  - MAIMLワークフローファイルの新規作成
  - サブフォルダ内でのファイル作成

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 モーダル構造

```
┌──────────────────────────────────────────────────┐
│ 📄 新規XMLファイル作成                    [×]    │
├──────────────────────────────────────────────────┤
│                                                  │
```

```
| ⓘ フォーマット情報                                |
|   フォーマット: material                          |
|   ファイルタイプ: XML                             |
|   拡張子: .xml                                    |
|                                                   |
| ファイル名 *                                      |
| ┌──────────────────────────────┬──────────┐      |
| | material_template_001        |  .xml    |      |
| |                              | (固定)   |      |
| └──────────────────────────────┴──────────┘      |
| 拡張子 .xml は自動で付加されます                  |
|                                                   |
|                  [キャンセル] [作成]              |
└──────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 モーダルヘッダー

- **アイコン**: FileAddOutlined（青色）
- **タイトル**:
  - XML: 「新規XMLファイル作成」
  - MAIML: 「新規MaiMLファイル作成」

2.2.2 エラー表示エリア（条件付き）

- **表示条件**: エラー発生時のみ
- **タイプ**: Alert（error、closable）
- **内容**: エラーメッセージ

2.2.3 フォーマット情報表示（条件付き）

- **表示条件**: formatNameが存在する場合
- **タイプ**: Alert（info）
- **内容**:
  - フォーマット名
  - ファイルタイプ（XML/MAIML）

- 拡張子（.xml/.maiml）

2.2.4 ファイル名入力フィールド

- **タイプ**: Input + Space.Compact
- **構成**:
  - 左側: ファイル名入力欄（可変幅）
  - 右側: 拡張子表示欄（固定幅80px、無効化）
- **バリデーション**:
  - 必須
  - 最小1文字
  - 最大100文字
- **機能**:
  - 文字数カウント表示
  - オートフォーカス
  - 補足説明（拡張子自動付与の説明）

2.2.5 アクションボタン

- **キャンセルボタン**: モーダルを閉じる
- **作成ボタン**: ファイル作成を実行（ローディング表示あり）

2.3 インタラクション

2.3.1 モーダル表示

- **操作**:「新規ファイル作成」ボタンをクリック
- **結果**: モーダルが開き、ファイル名入力フィールドにフォーカスが移動

2.3.2 ファイル名入力

- **操作**: ファイル名を入力
- **結果**:
  - 文字数がリアルタイムで表示される
  - 100文字以上は入力できない
  - 拡張子が右側に固定表示される

2.3.3 作成実行

- **操作**: 「作成」ボタンをクリック
- **結果**:
  - バリデーションが実行される
  - 成功時: モーダルが閉じ、ファイル一覧に新しいファイルが表示される
  - 失敗時: エラーメッセージが表示される

2.3.4 キャンセル

- **操作**: 「キャンセル」ボタンまたは×ボタンをクリック
- **結果**:
  - フォームがリセットされる
  - モーダルが閉じる

---

3. 機能仕様

3.1 ファイル名のサニタイズ

3.1.1 サニタイズ処理

```
const sanitizedFileName = values.fileName
  .replace(/\s+/g, '_')              // スペース → アンダースコア
```

```
    .replace(/[^a-zA-Z0-9_-]/g, '_');  // 英数字・アンダースコア・ハイフン以外 → アンダースコア
```

- **目的**: ファイルシステムで使用できない文字を除去
- **変換ルール**:
  - スペース → `_`
  - 英数字、`_`、`-` 以外 → `_`

3.1.2 拡張子付きファイル名生成

```
const extension = fileType ? `.${fileType}` : '';
const fileNameWithExtension = `${sanitizedFileName}${extension}`;
```

- **例**:
  - 入力: `material template 001`
  - サニタイズ後: `material_template_001`
  - 最終: `material_template_001.xml`

3.1.3 UI動作

- ユーザー操作: ファイル名を入力
- 画面変化:
  - 入力値がそのまま表示される
  - サニタイズは送信時に実行される（ユーザーには見えない）

3.2 テンプレートタイプ自動判定（XMLのみ）

3.2.1 判定ロジック

```
let templateType: 'material' | 'condition' | 'result' | undefined;
if (fileType === 'xml') {
  const folderName = (formatId || formatName || '').toLowerCase();
  if (folderName === 'material') {
    templateType = 'material';
  } else if (folderName === 'condition') {
    templateType = 'condition';
  } else if (folderName === 'result') {
    templateType = 'result';
  }

  // テンプレートタイプが判定できない場合はエラー
  if (!templateType) {
    throw new Error('XMLファイルはmaterial、condition、resultフォルダでのみ作成できます');
  }
}
```

- **判定基準**: フォーマットID（またはフォーマット名）を小文字に変換して判定
- **対応フォルダ**:
  - `material` → Material テンプレート
  - `condition` → Condition テンプレート
  - `result` → Result テンプレート
- **エラーケース**: 上記以外のフォルダでXMLファイルを作成しようとした場合

3.2.2 UI動作

- ユーザー操作:「作成」ボタンをクリック
- 画面変化:
  - XMLファイルの場合、テンプレートタイプが自動判定される
  - 判定できない場合、エラーメッセージが表示される
  - MAIMLファイルの場合、テンプレートタイプは `undefined`

3.3 バリデーション

3.3.1 必須チェック

```
{
  required: true,
  message: 'ファイル名を入力してください',
}
```

3.3.2 最小文字数チェック

```
{
  min: 1,
  message: 'ファイル名は1文字以上で入力してください',
}
```

3.3.3 最大文字数チェック

```
{
  max: 100,
  message: 'ファイル名は100文字以内で入力してください',
}
```

3.3.4 UI動作

- ユーザー操作: 「作成」ボタンをクリック
- 画面変化:
  - バリデーションエラーがある場合、フィールド下部に赤いエラーメッセージが表示される
  - すべてのバリデーションが通過すると、送信処理が実行される

3.4 ファイル作成処理

3.4.1 送信ハンドラ

```
const handleSubmit = async () => {
  try {
    setError(null);
    setSubmitting(true);

    // バリデーション
    const values = await form.validateFields();

    // ファイル名サニタイズ
    const sanitizedFileName = values.fileName
      .replace(/\s+/g, '_')
      .replace(/[^a-zA-Z0-9_-]/g, '_');

    const fileNameWithExtension = `${sanitizedFileName}${extension}`;

    // テンプレートタイプ判定（XMLのみ）
    let templateType: 'material' | 'condition' | 'result' | undefined;
    if (fileType === 'xml') {
      // ... 判定ロジック
    }

    // ファイルデータの作成
    const fileData: CreateFormatFileData = {
      fileName: values.fileName,
      fileNameWithExtension: fileNameWithExtension,
      formatId: formatId || '',
      formatName: formatName || '',
      fileType: fileType || 'xml',
      templateType: templateType,
      createdBy: createdBy,
      createdByName: createdByName,
      currentPath: currentPath.length > 0 ? currentPath.join('/') : undefined,
    };

    // 親コンポーネントの送信ハンドラを呼び出し
    await onSubmit(fileData);

    // 成功時はフォームをリセット
    form.resetFields();
  } catch (err) {
    // エラーハンドリング
  } finally {
    setSubmitting(false);
```

```
  }
};
```

3.4.2 UI動作

- ユーザー操作: 「作成」ボタンをクリック
- 画面変化:
  - ボタンがローディング状態になる
  - 成功時: モーダルが閉じ、フォームがリセットされる
  - 失敗時: エラーメッセージが表示される

3.5 エラーハンドリング

3.5.1 ファイル名重複エラー

```
if (errorMessage.includes('同じ名前のファイルが既に存在します') ||
    errorMessage.includes('already exists')) {
  form.setFields([
    {
      name: 'fileName',
      errors: [errorMessage],
    },
  ]);
}
```

- **表示場所**: ファイル名入力フィールドの下部
- **表示内容**: 「同じ名前のファイルが既に存在します」

3.5.2 テンプレートタイプ判定エラー

```
throw new Error('XMLファイルはmaterial、condition、resultフォルダでのみ作成できます');
```

- **表示場所**: モーダル上部のAlertコンポーネント

- **表示内容**: 「XMLファイルはmaterial、condition、resultフォルダでのみ作成できます」

3.5.3 その他のエラー

```
setError(errorMessage);
```

- **表示場所**: モーダル上部のAlertコンポーネント
- **表示内容**: エラーメッセージ（API返却値）

3.5.4 UI動作

- ユーザー操作: なし（エラー発生時に自動表示）
- 画面変化:
  - 重複エラー: ファイル名フィールドの下部に赤いエラーメッセージ
  - その他のエラー: モーダル上部にAlertコンポーネント（閉じるボタンあり）

3.6 キャンセル処理

3.6.1 キャンセルハンドラ

```
const handleCancel = () => {
  form.resetFields();
  setError(null);
  onCancel();
};
```

3.6.2 UI動作

- ユーザー操作: 「キャンセル」ボタンまたは×ボタンをクリック
- 画面変化:
  - フォームがリセットされる
  - エラーメッセージがクリアされる

- モーダルが閉じる

---

4. データ仕様

4.1 Props定義

```
interface CreateFormatFileModalProps {
  open: boolean;                                    // モーダル表示状態
  onCancel: () => void;                             // キャンセル時のコールバック
  onSubmit: (data: CreateFormatFileData) => Promise<void>; // 送信時のコールバック
  formatId: string | null;                          // フォーマットID
  formatName: string | null;                        // フォーマット名
  fileType: 'xml' | 'maiml' | null;                 // ファイルタイプ
  createdBy: string;                                // 作成者ユーザーID
  createdByName: string;                            // 作成者名
  currentPath?: string[];                           // 現在のフォルダパス（サブフォルダ階層）
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	モーダルの表示状態（true=表示、false=非表示）
onCancel	() => void	✅	キャンセル時に呼び出されるコールバック関数
onSubmit	(data: CreateFormatFileData) => Promise	✅	送信時に呼び出されるコールバック関数（非同期）
formatId	string | null	✅	フォーマットのID（テンプレートタイプ判定に使用）
formatName	string | null	✅	フォーマットの名前（表示とテンプレートタイプ判定に使用）
fileType	'xml' | 'maiml' | null	✅	ファイルタイプ（拡張子とテンプレートタイプ判定に使用）
createdBy	string	✅	作成者のユーザーID
createdByName	string	✅	作成者の名前

currentPath	string[]	-	現在のフォルダパス（配列形式、例: `['subfolder1', 'subfolder2']`）

4.2 CreateFormatFileData型定義

```
export interface CreateFormatFileData {
  fileName: string;                    // ユーザー入力のファイル名（拡張子なし）
  fileNameWithExtension: string;       // サニタイズ済みファイル名（拡張子付き）
  formatId: string;                    // フォーマットID
  formatName: string;                  // フォーマット名
  fileType: 'xml' | 'maiml';           // ファイルタイプ
  templateType?: 'material' | 'condition' | 'result'; // テンプレートタイプ（XMLのみ）
  createdBy: string;                   // 作成者ユーザーID
  createdByName: string;               // 作成者名
  currentPath?: string;                // 現在のパス（スラッシュ区切り文字列）
}
```

CreateFormatFileData説明:

フィールド名	型	必須	説明
fileName	string	✅	ユーザーが入力したファイル名（拡張子なし、サニタイズ前）
fileNameWithExtension	string	✅	サニタイズ済みファイル名（拡張子付き、例: `material_template_001.xml`）
formatId	string	✅	フォーマットのID
formatName	string	✅	フォーマットの名前
fileType	'xml' | 'maiml'	✅	ファイルタイプ
templateType	'material' | 'condition' | 'result'	-	テンプレートタイプ（XMLファイルの場合のみ）
createdBy	string	✅	作成者のユーザーID
createdByName	string	✅	作成者の名前
currentPath	string	-	現在のフォルダパス（スラッシュ区切り、例: `subfolder1/subfolder2`）

4.3 State定義

```
const [submitting, setSubmitting] = useState(false);  // 送信中フラグ
const [error, setError] = useState<string | null>(null); // エラーメッセージ
const [form] = Form.useForm();                        // Ant Design Form インスタンス
```

State説明:

State名	型	説明
submitting	boolean	送信処理中かどうか（true=送信中、ボタンがローディング状態になる）
error	string | null	エラーメッセージ（null=エラーなし）
form	FormInstance	Ant Design Formのインスタンス（バリデーションとリセットに使用）

---

5. API連携

このコンポーネントは直接APIを呼び出しません。 親コンポーネント（FormatManagement）の `onSubmit` コールバックを呼び出し、親コンポーネントがAPI通信を行います。

5.1 呼び出されるAPI（親コンポーネント経由）

想定されるAPI:

- **POST** `/api/v1/experiment_admin/formats/{formatId}/files`
  - ファイル作成API
  - リクエストボディ: `CreateFormatFileData`
  - レスポンス: 作成されたファイル情報

---

6. エラーハンドリング

6.1 エラーケースと表示方法

エラーケース	検知タイミング	表示方法	表示内容

ファイル名未入力	バリデーション時	フィールド下部	「ファイル名を入力してください」
ファイル名が短すぎる	バリデーション時	フィールド下部	「ファイル名は1文字以上で入力してください」
ファイル名が長すぎる	バリデーション時	フィールド下部	「ファイル名は100文字以内で入力してください」
ファイル名重複	API返却時	フィールド下部	「同じ名前のファイルが既に存在します」
テンプレートタイプ判定失敗	送信処理時	モーダル上部Alert	「XMLファイルはmaterial、condition、resultフォルダでのみ作成できます」
API通信エラー	API返却時	モーダル上部Alert	APIから返却されたエラーメッセージ

6.2 エラー回復方法

- **バリデーションエラー**: ユーザーが入力を修正
- **重複エラー**: ユーザーがファイル名を変更
- **テンプレートタイプ判定エラー**: ユーザーが適切なフォルダを選択してから再試行
- **API通信エラー**: ユーザーが再試行またはキャンセル

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/CreateFormatFileModal.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **コンテキスト**: `frontend/src/contexts/LanguageContext.tsx`
- **型定義**: このファイル内で定義・export（CreateFormatFileData型）

7.3 依存ライブラリ

- **Ant Design**:
  - Modal
  - Form
  - Input
  - Space
  - Typography (Text)
  - Alert
  - theme
- **Ant Design Icons**:
  - FileAddOutlined
- **Context**:
  - useLanguage（多言語対応）

7.4 親子関係

```
FormatManagement (親)
  ↓ Props: open, onCancel, onSubmit, formatId, formatName, fileType, createdBy, createdByName,
currentPath
CreateFormatFileModal (このコンポーネント)
```

親コンポーネントから受け取るデータ:

- モーダル表示状態
- フォーマット情報（ID、名前、ファイルタイプ）
- 作成者情報
- 現在のフォルダパス

親コンポーネントへの通知:

- キャンセルイベント（`onCancel()`）

- 送信イベント（`onSubmit(data)`）

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 600px（固定）
- **高さ**: コンテンツに応じて可変

8.2 アクセシビリティ

- **オートフォーカス**: モーダル表示時、ファイル名フィールドに自動フォーカス
- **キーボード操作**: Enterキーで送信、Escキーでキャンセル
- **エラーメッセージ**: 明確で具体的なメッセージ

8.3 視覚的フィードバック

8.3.1 ローディング状態

- 送信中は「作成」ボタンがローディング表示
- モーダルは閉じられない（confirmLoadingで制御）

8.3.2 エラー表示

- **フィールドエラー**: 赤い枠線 + 下部に赤いエラーメッセージ
- **一般エラー**: 上部にAlertコンポーネント（閉じるボタンあり）

8.3.3 情報表示

- **フォーマット情報**: 青いAlertコンポーネントで目立たせる
- **拡張子**: 右側に固定表示（無効化されたInput）

8.4 ユーザーガイダンス

- **補足説明**: 「拡張子 .xml は自動で付加されます」
- **文字数カウント**: リアルタイムで表示（例: `25/100`）
- **プレースホルダー**: 「ファイル名を入力」

8.5 多言語対応

- **LanguageContext**: `useLanguage` フックを使用
- **翻訳キー**:
  - `formatManagement.create.titleXml`
  - `formatManagement.create.titleMaiml`
  - `formatManagement.create.fileName`
  - `formatManagement.create.fileNameRequired`
  - など

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline